评分：_____

南京信息工程大学

《面向对象程序设计实践》课程

综合实训报告

题　目　Campus Rainhub

学　　院：计算机学院

班　　级：计科专业 6 班

小组编号：第 41 组

学　　号：202483290399

姓　　名：陈秋雯

任课教师：耿学华

学　　期：2025-2026（1）

Campus Rainhub

1 系统分析

1.1 应用系统简介

Campus RainHub（校园智能雨具共享系统）是一套面向校园场景的智能雨具共享管理系统，旨在解决学生在雨天被困教学楼的痛点问题。系统采用 C++17 和 Qt 6.9.3 开发，使用 MySQL 8.0 作为数据存储，提供用户端应用和管理员后台两个独立的应用系统。

系统核心思想是在校园内设置智能终端站点，学生可通过校园一卡通自助借还雨具，实现 24 小时无人值守服务。系统支持多种雨具类型（普通塑料伞、高质量抗风伞、遮阳伞、雨衣），每个站点配备 12 个槽位，按类型分配。客户端提供实时地图查看各站点位置和库存、账户管理、自助借还雨具等功能；管理员后台提供站点监控、雨具管理、用户管理、订单统计等运营管理功能。

1.2 系统需求（功能）分析

1.2.1 用户端功能需求

（1）用户认证模块

首次登录：新用户通过校园卡刷卡后需设置初始密码完成账户激活

用户登录：支持两种登录方式

校园卡刷卡登录：通过刷卡获取用户信息后输入密码

手动输入登录：手动输入用户 ID 和姓名后输入密码（因为系统是以软件形式模拟真实的终端，所以演示模式下采用手动输入账号和密码的方式）

密码管理：

修改密码：已登录用户可通过输入旧密码修改新密码

忘记密码：提示用户联系管理员进行密码重置

（2）借还雨具模块

借还页面每 3 秒自动刷新槽位状态

借雨具：

- 用户选择目标站点和槽位
- 系统验证用户余额、雨具可用性
- 完成借出后自动扣除费用并生成借还记录
- 支持查看站点各槽位的雨具类型和状态

还雨具：

- 用户选择归还站点和槽位
- 系统验证雨具归属关系（即归还槽位是否与归还雨具类型相符）
- 完成归还后更新雨具状态和站点库存

• 自动计算费用并更新用户余额

（3）实时地图模块

站点展示：在地图上以可视化方式显示所有站点位置，支持点击查看站点详情

库存查询：实时显示各站点各类雨具的可用数量，数据从数据库动态获取

状态显示：通过颜色标识显示站点库存状态（绿色：充足≥5，黄色：紧张 2-4，红色：不足<2）

站点信息：点击站点可查看站点名称、可借雨具数量、站点描述等详细信息

（4）账户管理模块

个人信息：查看用户 ID、姓名、角色（学生/教职工）

余额查看：查看当前账户余额，支持刷新余额显示最新数据

（5）使用说明模块

提供系统使用指南，帮助用户了解操作流程和注意事项

（6）主页仪表板模块

• 显示用户欢迎信息

• 提供快速入口：借伞、还伞、查看地图、个人中心、使用说明

• 站点选择下拉框，方便快速选择目标站点

1.2.2 管理端功能需求

（1）管理员认证模块

管理员登录：管理员通过管理员账号和密码登录后台系统，与用户端登录完全分离，确保权限隔离

（2）首页概览模块（Dashboard）

统计信息：

设备在线率：显示所有站点的在线率百分比

雨具借出数量：显示当前已借出的雨具总数

待处理故障：显示故障雨具数量

站点概览表格：显示所有站点的总雨具数、可借数量、已借出数量、故障数量

实时刷新：定时自动刷新数据（管理端是每 5 秒，用户端是每 3 秒），确保管理员获取最新状态

（3）站点监控模块

站点详情查看：查看每个站点的具体信息，包括位置坐标、在线状态、各槽位占用情况

（4）雨具管理模块

雨具查询：

• 按站点筛选查看所有雨具信息

• 按槽位筛选查看特定槽位的雨具

• 显示雨具 ID、类型、所在站点、槽位、状态等信息

雨具操作：

• 状态管理：修改雨具状态（可用/已借/故障）

（5）用户管理模块

用户列表：查看所有注册用户信息，包括用户 ID、姓名、角色、余额、账户状态

用户搜索：支持按用户 ID 或姓名搜索用户

重置用户密码：管理员可为用户重置密码

（6）订单统计模块

借还记录：查看所有借还记录，包括流水号、用户信息、雨具信息、借出时间、归还时间、费用等

1.2.3 系统非功能需求

（1）性能需求

系统响应时间：用户操作响应时间应在 1 秒内，确保良好的用户体验

并发支持：支持多用户同时使用。系统采用基于线程 ID 的连接管理机制：每个线程维护独立的数据库连接实例，连接名基于线程 ID 生成，确保连接隔离。在当前单线程应用场景下，每个进程的主线程拥有一个独立连接；多个终端（多个进程）可同时访问数据库，实现真正的并发支持。该设计也为未来可能的多线程扩展提供了良好的架构支持

数据实时性：地图库存数据实时更新，每次查询都从数据库读取最新数据，确保用户看到准确的可用数量

数据加载优化：地图页面的静态数据（站点名称、坐标、描述）从 JSON 配置文件读取，动态数据（库存数量）从数据库读取，实现快速加载和实时更新的平衡

（2）安全需求

用户认证：用户和管理员登录完全分离，管理员账号无法在用户端登录，防止权限混淆

密码安全：用户密码采用加密存储，首次登录需设置密码

数据完整性：借还操作通过数据库事务保证数据一致性，确保操作的原子性

权限隔离：用户端和管理端采用独立的认证机制，确保权限隔离

（3）可用性需求

界面友好：采用现代化 UI 设计，使用 QSS 样式表实现深蓝渐变主题，界面美观统一

操作简便：支持刷卡快速登录，减少用户操作步骤；提供清晰的操作指引

错误提示：提供清晰的错误提示信息，帮助用户理解操作结果和失败原因

响应式设计：界面布局合理，适配不同屏幕尺寸

（4）可维护性需求

代码结构：采用分层架构，职责清晰，便于维护和扩展

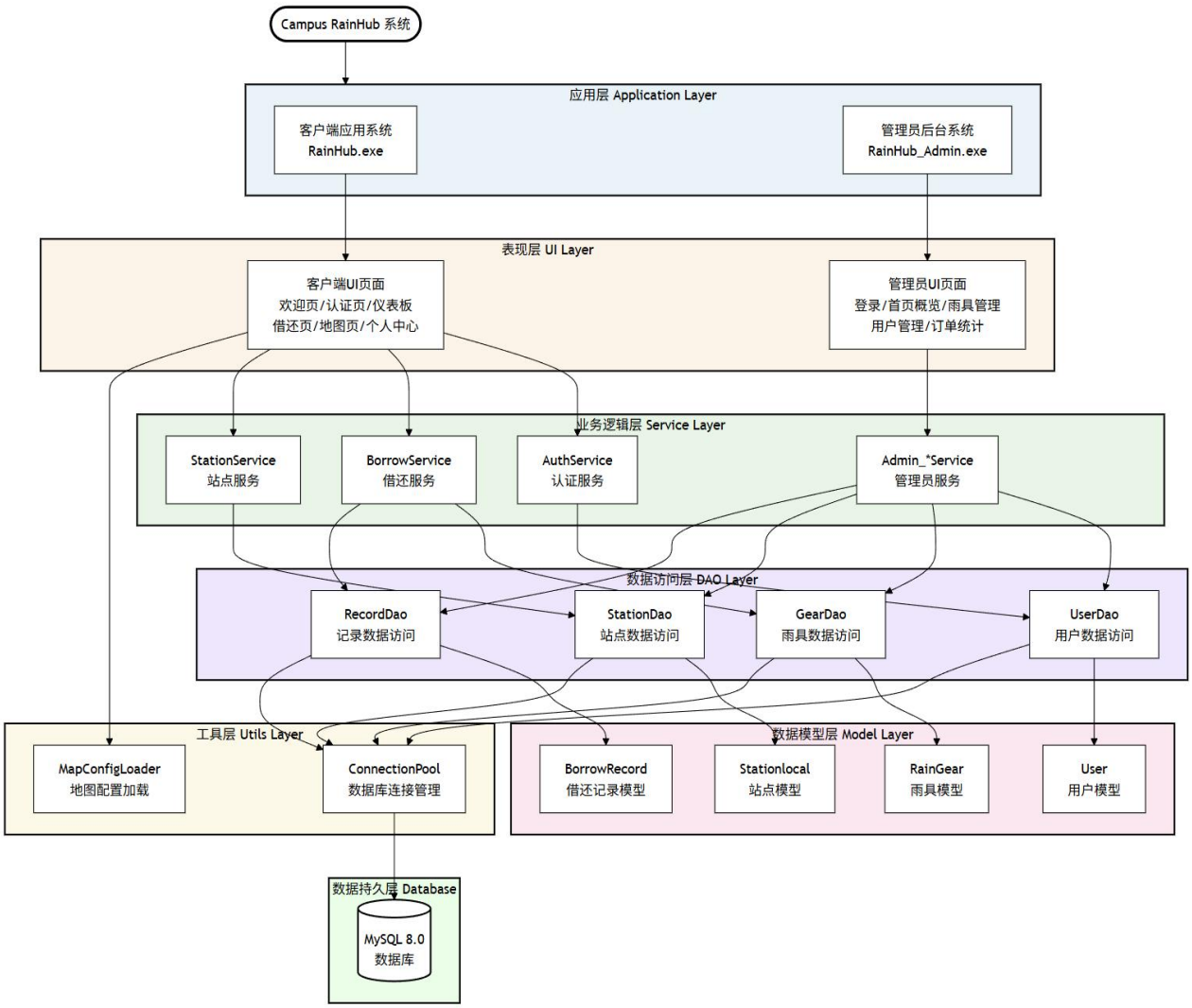
设计模式：使用工厂模式（雨具对象创建）、DAO 模式（数据访问抽象）等，提高代码复用性

配置管理：站点信息通过 JSON 配置文件管理，便于修改和扩展，无需修改代码即可更新站点信息

代码规范：遵循 C++编码规范，代码注释清晰，便于团队协作和后续维护

2 系统设计

2.1 总体设计



系统采用经典分层架构设计，自顶向下分为以下六层：

1 表现层（UI 层）

客户端 UI：MainWindow 统一管理，包含欢迎页、认证页（刷卡模拟/手动输入、首次激活、登录、修改密码）、仪表板、借还页（3 秒自动刷新槽位状态）、地图页（JSON 静态配置+数据库动态库存）、个人中心、使用说明等页面。采用 QSS 样式表实现现代化深蓝渐变主题界面。

管理员后台 UI：AdminMainWindow 包含登录页、首页概览（设备在线率/雨具借出数量/待处理故障/站点概览表格）、雨具管理（按站点和槽位筛选/状态修改）、用户管理（用户列表/搜索/重置密码）、订单统计（最近订单流水）等页面。5 秒定时刷新当前页面数据，确保管理员获取最新状态。

2 业务逻辑层（Control 层）

AuthService：提供 checkLogin、verifyPassword、activateUser 方法，负责用户认证、首次激活与密码更新。检查用户是否存在、姓名是否匹配、账户是否激活，实现用户端登录认证逻辑。

BorrowService：提供 borrowGear、returnGear 方法，负责借出与归还的核心业务逻辑。包含押金冻结、类型-槽位合法性校验、槽位占用检测、按小时计费（不足 1 小时按 1 小时计算）、押金退还计算与数据库事务管理，确保借还操作的原子性和数据一致性。

StationService：提供 getAllStations、getStationDetail、getStationInventoryCounts 方法，负责站点聚合查询与地图库存统计。支持获取所有站点完整信息、单个站点详情、各站点可用库存数量等查询功能。

Admin_Service：管理员端各类服务，包括 Admin_StationService（站点统计与在线率）、Admin_GearService（雨具查询与状态管理）、Admin_UserService（用户查询与管理）、Admin_OrderService（订单统计与查询）等，为管理员后台提供数据统计、查询与操作入口。

3 数据访问层（DAO 层）

UserDao：提供 selectById、selectByIdAndName、updatePassword、updateBalance、selectAll 方法，封装用户数据的查询、密码更新、余额更新等操作。

GearDao：提供 selectById、selectByStation、isSlotOccupied、insert、deleteById、updateStatusAndLocation、updateStatus、selectAllDTO、countByStatus 方法，封装雨具的查询、插入、删除、状态更新、槽位占用检测以及管理员后台的 DTO 查询和统计功能。

StationDao：提供 selectAll、selectById、selectStationInventoryCounts、selectAllWithStats、getOnlineRate、updateStationStatus 方法，封装站点的查询、库存统计、站点统计信息查询、在线率计算以及站点状态更新等操作。

RecordDao：提供 addBorrowRecord、selectUnfinishedByUserId、updateReturnInfo、selectRecent 方法，封装借还记录的添加、未归还记录查询、归还信息更新以及管理员后台的最近订单查询功能。

4 数据模型层（Model 层）

RainGear 抽象类与子类：采用工厂模式和继承实现多态，RainGear 抽象基类定义雨具的共有属性和行为，RainGear_subclasses 实现具体的雨具子类（普通塑料伞、高质量抗风伞、专用遮阳伞、雨衣），RainGearFactory 提供工厂方法创建不同类型的雨具对象。每个子类实

现 get_deposit 和 get_iconpath 方法，实现押金与图标的多态。

Stationlocal：站点模型类，维护 12 个槽位的库存信息、可用/故障状态与位置坐标。提供添加雨具、取出雨具、标记槽位可用/不可用、查询库存数量等方法。

BorrowRecord：借还记录实体类，封装记录 ID、用户 ID、雨具 ID、借出时间、归还时间、费用等字段。

User：用户模型类，封装用户 ID、姓名、密码、角色、余额、激活状态等属性。

全局枚举：GlobalEnum.hpp 定义雨具类型枚举（GearType）、雨具状态枚举（GearStatus）、站点枚举（Station），提高代码可读性和类型安全。

5 工具层（Utils 层）

ConnectionPool：线程本地数据库连接管理，基于线程 ID 生成连接名，每个线程维护独立的数据库连接实例，避免并发冲突。提供 getThreadLocalConnection 和 removeThreadConnection 方法。

MapConfigLoader：地图静态配置加载器，从 JSON 资源文件（:/map/map_config.json）读取站点静态信息（站点 ID、名称、坐标、描述），实现快速加载。与数据库动态库存查询结合，实现"快加载+实时更新"的平衡。

6 数据持久层（MySQL）

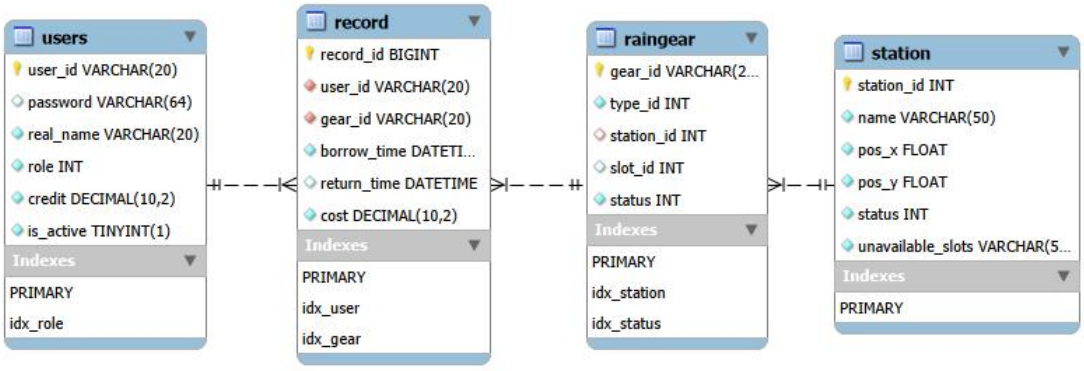
存储用户、站点、雨具、借还记录等所有业务数据。数据库初始化脚本位于 sql/init_db.sql，包含 4 个核心数据表：users（用户表）、station（站点表）、raingear（雨具表）、record（借还记录表）。采用 InnoDB 存储引擎，支持事务和外键约束，确保数据完整性和一致性。

2.2 详细设计

2.2.1 数据结构设计

应用系统开发需要的数据组织方式主要包括数据库表结构和程序内部数据结构两部分。

数据库表结构：



1. 用户表：用于存储学生、教职工及管理员信息。

核心功能：通过 role 字段实现多级权限管理（0:学生, 1:教职, 9:管理员）；通过 is_active 支持首次登录激活机制。

在 role 字段建立索引以优化权限校验速度；password 字段预留加密存储空间，确保数据安全。

2. 站点表：存储借还站点的地理位置与运行状态。

核心功能：unavailable_slots 采用逗号分隔符记录故障槽位，实现精细化运维。

station_id 严格对应程序中的 Station 枚举值，保持代码逻辑与数据存储的高度一致。

3. 雨具表：记录雨具的种类、实时位置及物理状态。

核心功能：基于 RFID 标签定义 gear_id；通过 slot_id（1-12）实现物理槽位锁定。

业务逻辑：槽位分区存放不同类型雨具（如 1-4 号存放普通伞），由程序根据 type_id 自动控制电机开启逻辑。

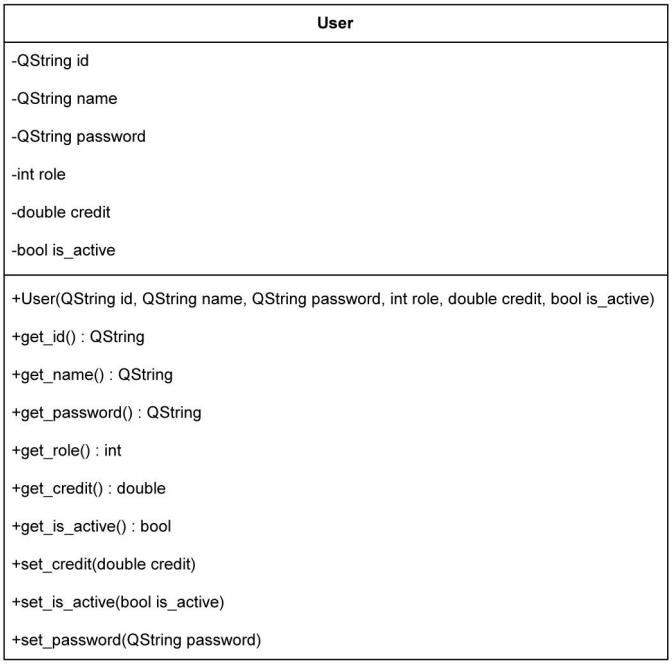
4. 借还记录表：追踪用户借还行为全生命周期，用于计费与统计。

核心功能：记录借还精确时间；return_time 为空时代表订单进行中，归还时由 BorrowService 自动计算 cost 费用。

2.2.2 主要类设计

Model 层：

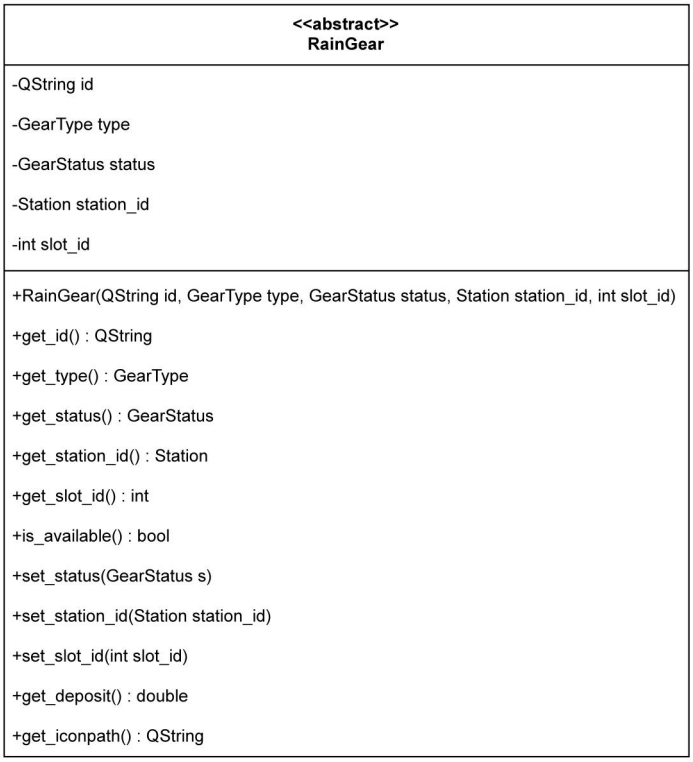
1.User 类

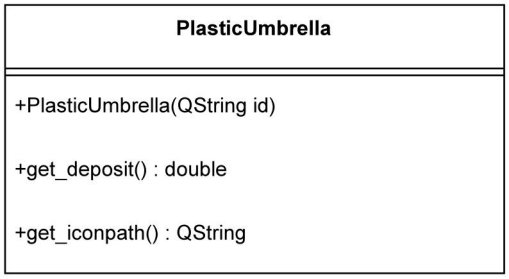


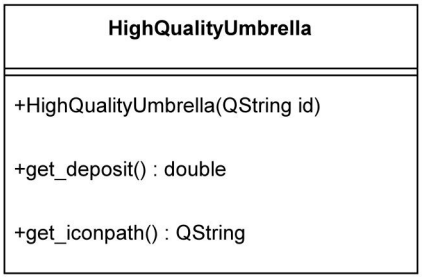
封装用户的基本信息，用于业务逻辑层和 UI 层的数据传递。功能：封装用户 ID、姓名、

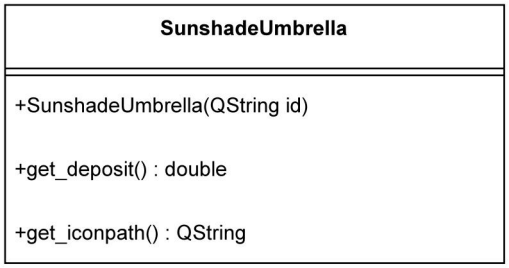
密码、角色、余额、激活状态等属性，提供 getter 和 setter 方法，支持余额更新和密码修改。主要接口包括：get_id()返回用户 ID，get_name()返回用户姓名，get_password()返回密码，get_role()返回角色，get_credit()返回账户余额，get_is_active()返回激活状态，set_credit(double) 设置账户余额， set_is_active(bool) 设置激活状态，set_password(QString&&)设置密码。

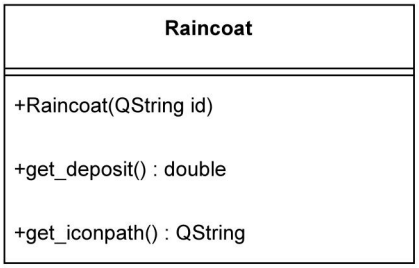
2.RainGear 雨具抽象基类及其派生类











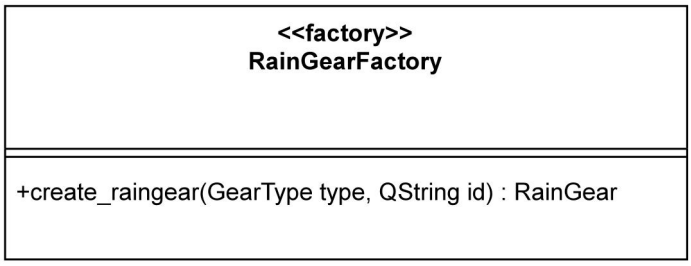
RainGear 是雨具的抽象基类，采用工厂模式和继承实现多态，定义了雨具的共有属性和行为。功能：封装雨具的基本信息（ID、类型、状态、位置），提供状态查询和位置更新接口，通过纯虚函数实现不同雨具类型的押金和图标多态。

主要接口包括：get_id()返回雨具 ID，get_type()返回雨具类型，get_status()返回雨具状态，is_available()判断雨具是否可借（状态为 Available），get_deposit()为纯虚函数获取押金金额，get_iconpath()为纯虚函数获取图标资源路径，set_status(GearStatus)设置雨具状态，set_station_id(Station)设置所在站点，set_slot_id(int)设置所在槽位。

算法流程：RainGear 作为抽象基类，不直接实例化。

其四个子类（PlasticUmbrella、HighQualityUmbrella、SunshadeUmbrella、Raincoat）分别重写 get_deposit 和 get_iconpath 方法，实现不同押金（10 元、20 元、15 元、25 元）和图标路径的返回。

3.RainGearFactory 雨具工厂类



RainGearFactory 是雨具工厂类，采用工厂模式创建不同类型的雨具实例。

功能：根据雨具类型枚举值，创建对应的雨具子类实例，封装对象创建逻辑，降低代码耦合度。

主要接口：create_raingear(GearType type, const QString &id)为静态工厂方法，根据类型和 ID 创建雨具实例。

算法流程：工厂方法内部使用 switch 语句，根据 GearType 枚举值匹配对应的雨具类型，调用相应的子类构造函数，返回 std::unique_ptr<RainGear>智能指针。如果类型未知，返回 nullptr。

4.Stationlocal 站点类

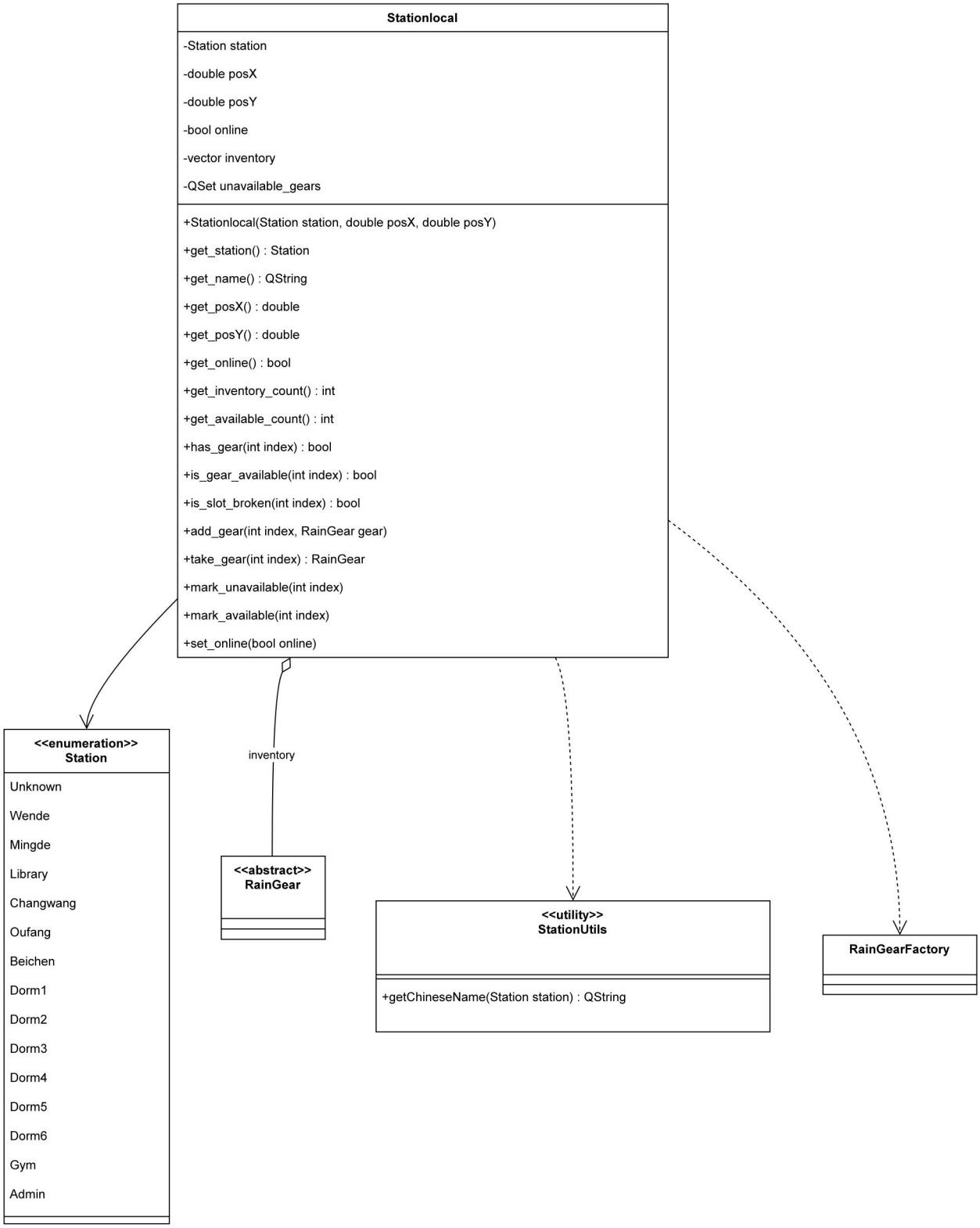
Stationlocal 是 Model 层的核心实体类，用于在客户端封装物理站点的静态属性与动态库存状态。核心功能信息封装：维护站点枚举 ID、地理坐标(x,y)及实时在线状态。

库存管理：管理 12 个物理槽位的雨具对象，支持雨具的添加、标记故障及状态同步。

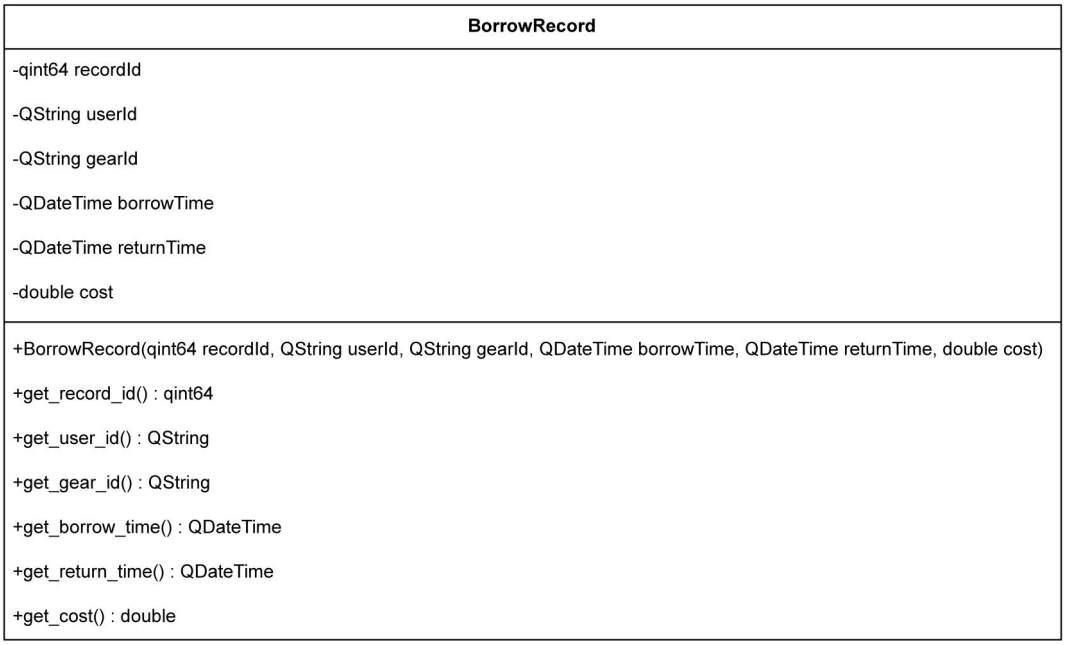
业务逻辑查询：提供总库存统计、可用雨具计算及槽位状态（是否有雨具、是否故障、是否可借）的综合判断。

算法与实现亮点内存安全：采用 std::vector<std::unique_ptr<RainGear>> 存储雨具对象，

通过智能指针实现槽位资源的自动管理。高效检索：使用 QSet<int> 维护故障槽位名单，实现 O（1）复杂度的槽位可用性校验。多重状态校验：is_gear_available 接口同步检查“槽位有伞”、“雨具状态为 Available”及“槽位非故障”三个条件，确保业务逻辑的严谨性。



5.BorrowRecord 类



BorrowRecord 是 Model 层 的数据实体类，主要用于在业务逻辑层与表现层之间封装和传递借还行为的完整数据快照。

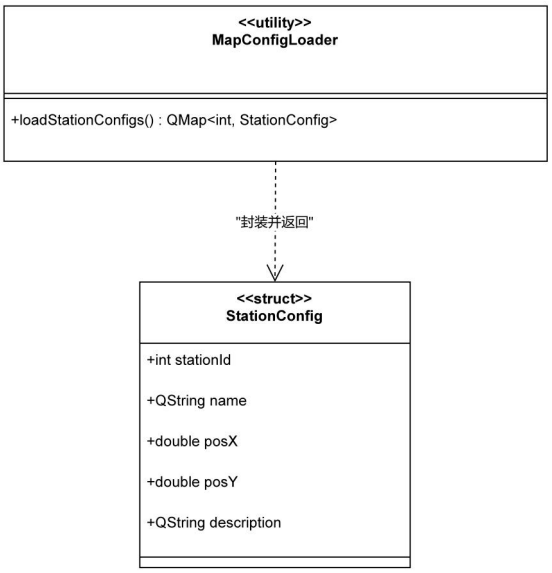
字段封装：严格对应数据库 record 表结构，涵盖记录 ID、用户/雨具 ID、借还时间戳及费用字段。

接口支持：提供全套 Getter 接口，支持 BorrowService 进行费用结算逻辑以及 UI 层的历史订单列表展示。

实例化机制：构造函数支持全参数初始化，便于 RecordDao 在执行数据库查询后将结果集快速转化为内存对象。

Utils 层：

1 地图加载器类



MapConfigLoader 负责从 JSON 资源文件中读取站点的静态地理信息，是系统实现“动静数据分离”的关键组件。

核心功能

静态配置读取：从内置资源路径 :/map/map_config.json 加载站点 ID、名称、坐标及描述等不经常变动的数据。

结构化映射：将原始 JSON 数据解析并映射为自定义的 StationConfig 结构体，并以 QMap 形式存储，支持 O(1) 复杂度的快速查找。

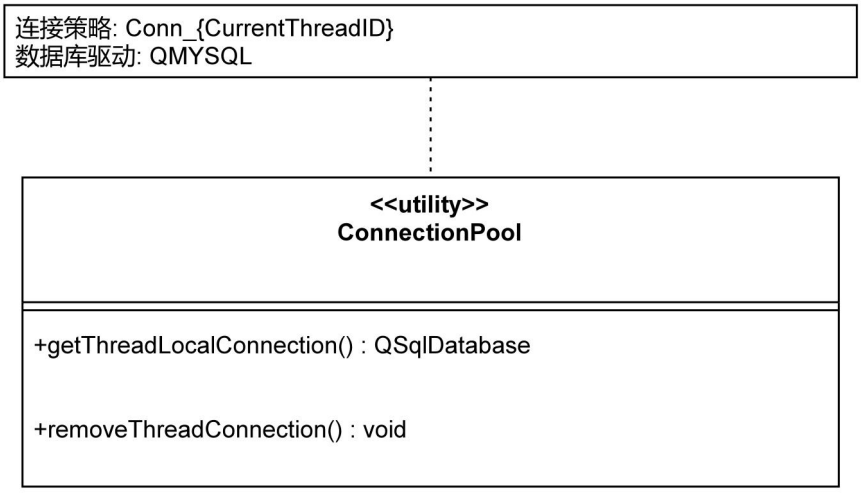
设计价值

性能优化：坐标等静态数据无需频繁查询数据库，直接从内存加载，大幅提升了地图界面的渲染效率。

动静分离架构：静态数据由 JSON 维护，动态库存由数据库维护，降低了 MySQL 的并发读写压力。

解耦设计：采用静态工具方法 loadStationConfigs，无需实例化对象即可完成配置加载，符合工具类设计规范。

2 数据库连接池类



核心功能：

线程隔离管理：通过 QThread::currentThreadId()为每个主线程创建唯一的连接标识（如 Conn_线程 ID），解决 QSqlDatabase 实例不能跨线程使用的限制。

连接生命周期维护：提供 getThreadLocalConnection 获取或创建当前线程连接，以及 removeThreadConnection 显式销毁连接，防止数据库产生过多的僵尸进程。

参数预设：统一封装 MySQL 8.0 的连接参数（Host、Port、User、Password），简化 DAO 层的调用复杂度。

补充：

1 数据访问层（DAO Layer）

数据访问层采用 DAO 模式，将数据库操作封装在独立的 DAO 类中，实现数据访问的抽象。本系统包含四个核心 DAO 类：UserDao 封装用户数据的查询、密码更新、余额更新等操作；GearDao 提供雨具的 CRUD 操作，支持按 ID 查询、按站点查询、槽位占用检测、状态和位置更新；StationDao 负责站点的查询操作，支持获取所有站点完整信息、单个站点详情、各站点可用库存数量统计；RecordDao 处理借还记录的添加、查询、更新等操作。所有 DAO 类都通过 ConnectionPool 获取数据库连接，使用参数化查询防止 SQL 注入。

2 业务逻辑 control 层（Service Layer）

业务逻辑层实现系统的核心业务逻辑，协调 DAO 层完成复杂业务流程。BorrowService 类实现借还雨具的核心业务，包含 borrowGear 和 returnGear 两个核心方法，实现借伞和还伞的完整业务流程，包括用户验证、余额检查、雨具状态更新、费用计算、数据库事务管理等，确保借还操作的原子性和数据一致性。AuthService 类实现用户认证、首次激活、密码验证等业务逻辑，提供 checkLogin、verifyPassword、activateUser 等方法。StationService 类提供站点相关的业务逻辑，包括获取所有站点信息、单个站点详情、各站点可用库存数量等功能。管理员后台还包含 Admin_StationService、Admin_GearService、Admin_UserService、Admin_OrderService 等服务类，分别处理站点管理、雨具管理、用户管理、订单统计等管理功能。

3 系统实现

3.1 系统开发环境

分类	技术详情
开发语言与标准	C++17
GUI 框架	Qt 6.9.3
数据库系统	MySQL 8.0
构建系统	CMake 3.16+
操作系统	Windows 10/11
编译器/工具链	MinGW (GCC 11.0+)
版本控制	Git
第三方依赖库	Qt Core, Qt Widgets, Qt Sql, MySQL Connector/C++

3.2 程序编码

本系统的程序实现严格遵循前述的六层架构设计，采用 C++17 标准与 Qt 6 框架进行开发。在编码过程中，核心业务逻辑集中于业务逻辑层（Service Layer），通过调用数据访问层（DAO Layer）提供的接口，实现对 MySQL 8.0 数据库的安全、高效操作。

3.2.1 借伞业务逻辑实现

借伞业务（borrowGear）是系统的核心逻辑之一，其编码实现重点在于多重前置校验与数据库事务的完整性。

首先，程序通过工具层（Utils）的 ConnectionPool 获取当前线程的数据库连接。在逻辑处理阶段，系统依次执行以下校验：首先调用 UserDao 检查用户是否存在、账户是否激活以及是否存在尚未归还的逾期订单；其次调用 GearDao 核实目标雨具的状态是否为“可用（Available）”；最后计算雨具对应的押金，并对比用户账户余额是否充裕。

在所有校验通过后，程序开启数据库事务（db.transaction()）。通过联动更新 users 表的余额、raingear 表的状态（置为 Borrowed）以及在 record 表中新增借还记录，确保了借伞操作的原子性。若中途任何环节出错，系统将执行回滚操作，以保证数据的一致性。

```
//借伞业务逻辑，传入用户ID和雨具ID
ServiceResult BorrowService::borrowGear(const QString& userId, const QString& gearId) {
    auto db = ConnectionPool::getThreadLocalConnection();
    if (!db.isOpen()) return {false,"数据库连接失败"};

    //检查用户是否存在&激活
    auto userBox=userDao.selectById(db, userId);
    if (!userBox) return {false,"用户不存在"};
    if (!userBox->get_is_active()) return {false, "账户未激活，请先去激活"};
    //检查用户是否已经借伞
    auto unfinishedRecord = recordDao.selectUnfinishedByUserId(db, userId);
    if (unfinishedRecord.has_value()) { return {false, "您有未归还的订单，请先归还后再借"}; }

    //检查雨具是否存在
    auto gear = gearDao.selectById(db, gearId);
    if(!gear) return {false,"雨具识别错误(ID不存在)"};
    //检查雨具状态
    if(gear->get_status()!= GearStatus::Available){ return {false,"该雨具已被借出或处于维护中"}; }

    //检查余额是否足够支付押金
    double deposit=gear->get_deposit();
    if (userBox->get_credit()<deposit){
        return {false, QString("余额不足，当前雨具需押金 %1 元").arg(deposit)};
    }

    if (!db.transaction()) return {false, "事务开启失败"};
    bool success = true;
    if (!userDao.updateBalance(db, userId, -deposit)) {
        qCritical() << "借伞失败：扣款步骤出错";
        success = false;
    }

    //更新雨具状态为借出
    if (success && !gearDao.updateStatusAndLocation(db, gearId, GearStatus::Borrowed, Station::Unknown,
0)) {
        qCritical() << "借伞失败：更新雨具状态出错";
        success = false;
    }
    //插入借出记录 (Record)
    if (success && !recordDao.addBorrowRecord(db, userId, gearId)) {
        qCritical() << "借伞失败：创建订单记录出错";
        success = false;
    }

    if (success) {
        db.commit();
        qInfo() <<"用户"<< userId <<"成功借出雨具"<<gearId;
        return {true, "借伞成功！请取走您的雨具"};
    } else {
        db.rollback();
        return {false, "系统内部错误，交易已取消"};
    }
}
```

3.2.2 还伞业务逻辑实现

还伞业务（returnGear）是系统资产管理与财务结算的核心闭环。该模块不仅涉及雨具物理位置的回收，还包含复杂的区域合法性校验与基于阶梯费率的自动计费算法。

在编码实现中，系统首先执行物理层面的“归位校验”。程序通过 gearDao 获取雨具详情，并利用 switch-case 结构对雨具类型（GearType）与目标槽位 ID（slotId）进行区间匹配校验。例如，普通塑料伞仅允许归还至 1-4 号槽位，高质量抗风伞对应 5-8 号槽位，这种设计确保了雨具在物理基站中的分区存储与规范化管理。同时，系统会调用 isSlotOccupied 接口检查目标槽位占用情况，从源头上避免物理冲突。

在业务逻辑校验阶段，程序会检索该用户的未完成订单。通过比对 record 表中记录的雨具 ID 与实际归还的雨具 ID，确保“借还一致性”。随后，系统进入自动计费流程。计费辅助函数 calculateCost 采用小时计费制，通过 qCeil 函数实现“不足一小时按一小时计算”的业务规则。根据不同雨具类型的单价（如抗风伞 2.0 元/小时），计算出实际产生租金。为了保障用户权益，系统设有封顶机制，即单次租金最大值不会超过其预付的押金金额。

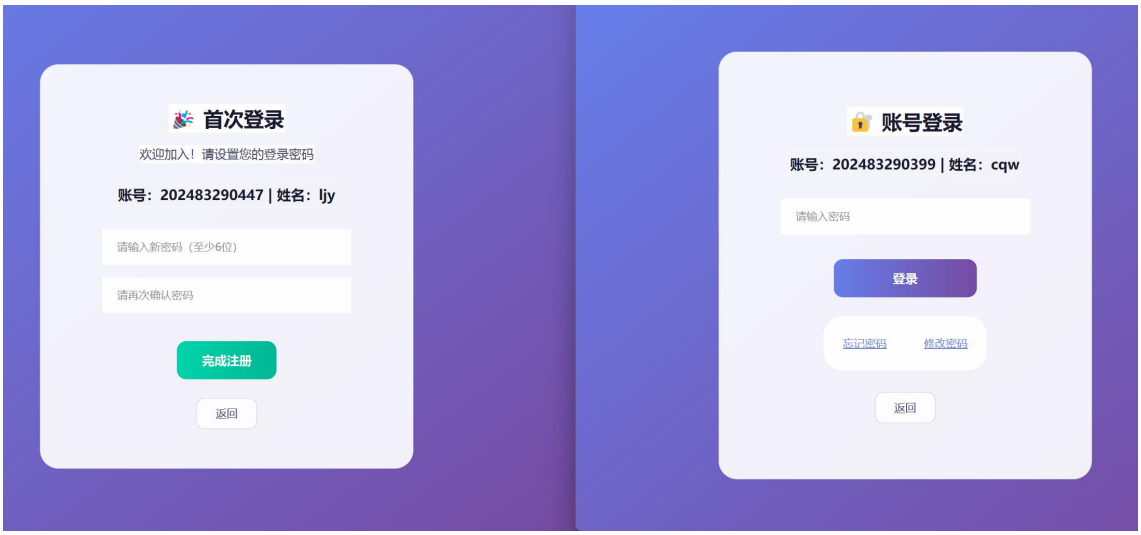
最后，系统通过数据库事务（db.transaction()）确保财务结算的安全性。程序同步更新 record 表的归还信息、raingear 表的状态位置，并计算应退金额（$refund = deposit - cost$），调用 updateBalance 接口将余额退回用户账户。一旦事务提交成功，系统将向表现层反馈详细的结算账单，实现业务流程的完整闭环。

```
//辅助函数：计费规则
double BorrowService::calculateCost(const QDateTime& borrowTime, const QDateTime& returnTime, GearType
type) {
    //计算秒数差
    qint64 seconds = borrowTime.secsTo(returnTime);
    if (seconds <= 0) return 0.0;
    //转换成小时
    double hours = qCeil(seconds / 3600.0);
    //根据类型定单价
    double rate = 1.0; // 默认 1元/小时
    switch (type) {
        case GearType::StandardPlastic: rate = 1.0; break;
        case GearType::PremiumWindproof: rate = 2.0; break;
        case GearType::Sunshade: rate = 1.5; break;
        case GearType::Raincoat: rate = 2.0; break;
        default: rate = 1.0; break;
    }
    //计算总价
    double total = hours * rate;
    return total;
}
```

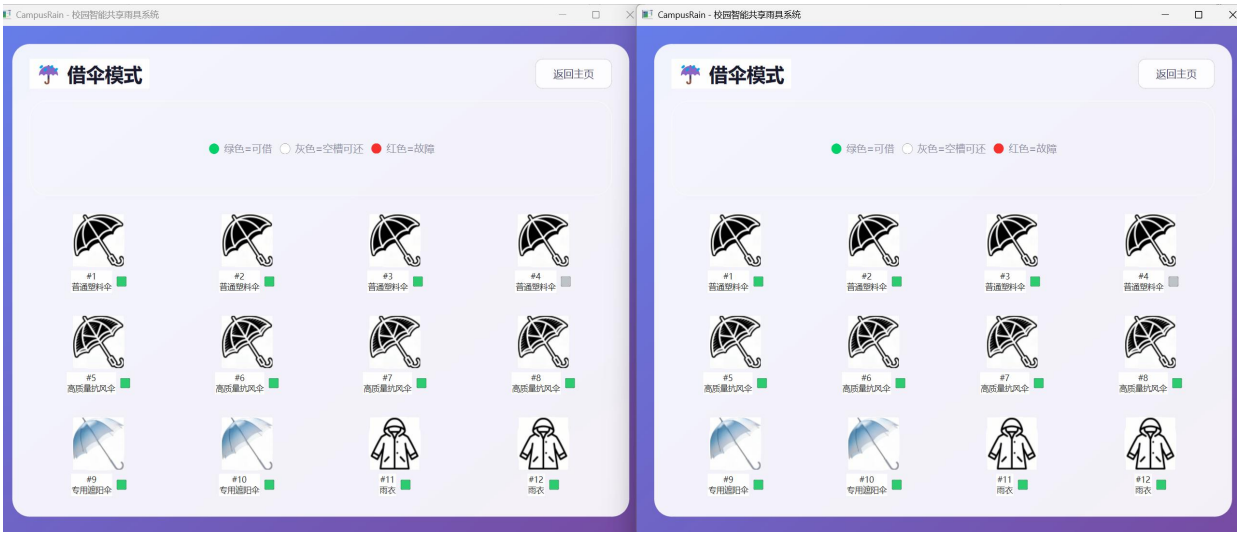
```
//还伞业务逻辑，传入用户ID和雨具ID，站点ID和槽位ID
ServiceResult BorrowService::returnGear(const QString& userId, const QString& gearId, Station stationId,
int slotId) {
    auto db = ConnectionPool::getThreadLocalConnection();
    if (!db.isOpen()) return {false, "数据库连接失败"};

    //添加雨具对应槽位的判断
    auto gear = gearDao.selectById(db, gearId);
    if (!gear) return {false, "雨具信息异常(ID不存在)"};
    bool isSlotValid = false;
    GearType type = gear->get_type();
    //根据雨具类型判断槽位是否合法
    switch (type) {
        case GearType::StandardPlastic:   //普通塑料伞: 1-4
            if (slotId >= 1 && slotId <= 4) isSlotValid = true;
            break;
        case GearType::PremiumWindproof:  //高质量抗风伞: 5-8
            if (slotId >= 5 && slotId <= 8) isSlotValid = true;
            break;
        case GearType::Sunshade:          //专用遮阳伞: 9-10
            if (slotId >= 9 && slotId <= 10) isSlotValid = true;
            break;
        case GearType::Raincoat:          //雨衣: 11-12
            if (slotId >= 11 && slotId <= 12) isSlotValid = true;
            break;
        default:
            break;
    }

    if (!isSlotValid) {return {false, "归还位置错误！该类型雨具只能还到指定区域（请查看槽位说明）"};}

    //检查归还的槽位是否已经被占了
    if (gearDao.isSlotOccupied(db, stationId, slotId)) { return {false, "该槽位已有雨具，请更换槽位"}; }

    //查找该用户的未归还订单
    auto recordBox = recordDao.selectUnfinishedByUserId(db, userId);
    if (!recordBox.has_value()) { return {false, "未查询到您的借出记录"}; }
    //校验归还的伞是否和借的一样
    if (recordBox->get_gear_id() != gearId) {
        return {false, "归还的雨具ID与订单不符"};
    }

    if (!gear) return {false,"雨具信息异常"};
    QDateTime borrowTime = recordBox->get_borrow_time();
    QDateTime returnTime = QDateTime::currentDateTime();

    //计算租金
    double cost = calculateCost(borrowTime, returnTime, gear->get_type());
    double deposit = gear->get_deposit();

    //费用最多等于押金，不会倒扣用户余额
    if (cost > deposit) { cost = deposit;}

    //应退金额 = 押金 - 费用（最低为0）
    double refund = deposit - cost;

    if (!db.transaction()) return {false, "系统忙"};
    bool success = true;
    //填入归还时间和费用
    if (!recordDao.updateReturnInfo(db, recordBox->get_record_id(), cost)) { success = false; }
    //更新雨具状态为可用
    if (success && !gearDao.updateStatusAndLocation(db, gearId, GearStatus::Available, stationId,
slotId)) { success = false; }
    //退还押金（扣除租金后的余额）
    if (success && !userDao.updateBalance(db, userId, refund)) { success = false; }

    if (success) {
        db.commit();
        QString msg = QString("还伞成功！产生费用 %1 元，退回 %2 元").arg(cost, 0, 'f', 2).arg(refund, 0, 'f',
2);
        qInfo() << msg;
        return {true, msg, cost};
    } else {
        db.rollback();
        return {false, "还伞失败，系统回滚"};
    }
}
```

4 系统测试

1 测试首次登录的用户是否会跳转到注册页面，非首次登录的用户直接跳转到输入密码页面。左图为使用一个未激活的用户账号登录，右图为使用已经激活并设置过密码后的账号登录，对应的页面均成功跳转。



2 测试多终端用户登录时数据是否实时同步，此时图中两名用户均处在明德楼站点借伞，左图用户借了四号槽位的雨具后，雨具的状态显示器由绿色变为灰色，右图用户页面实时刷新，4 号槽位的状态显示器也变为灰色。



3 测试管理员后台与用户端页面是否实时同步，手动在管理员后台页面上将明德楼的 1-4 槽位雨具全部设置为故障，此时处于明德楼站点的用户终端实时同步显示 1-4 槽位为故障。

4 判断归还雨具时槽位是否对应，无法将雨具还到类型不对应的槽位上

5 总结与体会

遇到的问题及解决办法：

我在这次后端开发中遇到的大多数问题都可以归根于大杂烩式的设计方式，可能刚开始写代码的时候只是想着从流程的角度一步步把功能完善，这次开发的初始阶段没有好好做架构，写得有点”随便”，这也是后面让我让对后端代码和结构进行重构的原因，因为 Model 层我写得比较早，当时也没想着这会成为我课程设计的选题，最初我的 User 类里面什么都有，有 setters/getters，还有比如像这种扣款的函数，其实时间隔久了我自己都快忘记当时怎么写的了，只记得很”随便”，不过正好当时上传了 GitHub，翻了一下 commit 想起来最开始怎么写的了

```
bool User::deduct(double amount) {
    if(credit>=amount){
        credit-=amount;
        return true;
    }
    return false;
} //这是我当时写的扣款的函数
```

然后，我当时的 control 层就两个类，一个是数据库连接池类（也就是对应的现在系统中的 utils/ConnectionPool.h），一个是 DataManger 类，其实就是把 sql 查询语句+业务逻辑中的条件判断+Model 层的接口调用全部放在一起了，其实这种方式比分层解耦的写法代码量会小很多，但功能不断变多之后就会有点乱，有时候看着之前自己写的注释也乱，所以其实没写多久之后就开始加入 dao 层进行重构了，不得不说把业务逻辑和 sql 查询这些分开之后代码量倒是增加不少，但是层层调用的规律其实都差不多，多写几个功能也就熟练了，后面的开发倒是没遇到什么大问题，还有一个让我印象比较深刻的就是 QVector 和 std::unique_ptr，这两个是不能一起使用的不过我一开始不知道 Qt 的容器（如 QVector, QString, QList）为了提高性能，普遍采用了“隐式共享”技术。意味着当拷贝一个 QVector 时，它并不会立即复制内存，而是让两个对象指向同一个数据块。也就是说存储在 Qt 容器中的元素必须是可拷贝的，这跟 std::unique_ptr 的用法就冲突了，当时使用 std::move，产生编译错误，后面就改回了使用 std::vector，然后 QMap 也用了一些，像比如那个将从 JSON 读取的站点 ID 作为 Key，将 StationConfig 结构体作为 Value 存储。

优/缺点

优点：整体功能相对比较完善，支持多终端。

缺点：有些细节前后端没对接好，比如管理员后台本来的设计是不仅可以手动修改雨具槽位的可借/故障状态，还能修改站点的在线状态的，因为 is_online 这个 status 字段数据库表中有，对应的类中也有属性，修改站点状态的函数也有，不过因为当时没对接好就导致了前

端页面没有这部分，只显示了一个站点在线率，对于这一点在我以后的合作开发中需要多注意。还有就是管理端在加载所有雨具的时候有一点点慢，这个后期应该考虑用视图优化。

体会&收获

体会是以后多人合作时要好好对接口，也要好好写注释，更要好好做架构。收获是让我了解了 Qt 这个框架，我还有一个还在构思的系统准备用 pyQt 开发，这次打了一些基础，但还需要不断学习。

然后其实也是非常敬佩我的队友，我一直觉得一个软件前端相当重要，因为前端是一个软件最直接的展示，我的队友在页面的设计上用了 QSS，让我们整个系统 UI 看上去很精美，比较有现代软件的感觉，没加之前的页面就显得有点点老了。

参考文献：

[1] C++ Primer

[2] Qt 官方文档

[3] 《Effective C++: 55 Specific Ways to Improve Your Programs and Designs》

小组编号	6 班第 41 组	组长	202483290399 陈秋雯
学号	姓名	具体任务（分工）	
202483290399	陈秋雯	系统后端	
202383290032	吴悠	系统前端	